

# Clautolisp User Manual

Codex

June 24, 2026

## Contents

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Scope</b>                                                | <b>1</b>  |
| <b>2</b> | <b>Synopsis</b>                                             | <b>1</b>  |
| <b>3</b> | <b>Modes of Operation</b>                                   | <b>2</b>  |
| 3.1      | File mode . . . . .                                         | 2         |
| 3.2      | Expression mode . . . . .                                   | 2         |
| 3.3      | Interactive REPL . . . . .                                  | 2         |
| <b>4</b> | <b>Options</b>                                              | <b>3</b>  |
| 4.1      | Dialect selection . . . . .                                 | 3         |
| 4.2      | Input selection . . . . .                                   | 4         |
| 4.3      | REPL behaviour . . . . .                                    | 4         |
| 4.4      | Host backend . . . . .                                      | 4         |
| 4.5      | Colour output . . . . .                                     | 4         |
| 4.6      | Source-file and terminal encoding . . . . .                 | 5         |
| 4.7      | Informational . . . . .                                     | 7         |
| <b>5</b> | <b>Image-introspection variables (*AUTOLISP-...*)</b>       | <b>7</b>  |
| 5.1      | Static — bound once, before the first user action . . . . . | 7         |
| 5.2      | Dynamic — bound only while the matching action runs . . . . | 9         |
| <b>6</b> | <b>Banner</b>                                               | <b>9</b>  |
| <b>7</b> | <b>Exit Codes</b>                                           | <b>9</b>  |
| <b>8</b> | <b>Errors and Diagnostics</b>                               | <b>10</b> |
| 8.1      | Encoding-dispatch diagnostics ( <b>enc-*</b> ) . . . . .    | 10        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>9 Examples</b>                                                   | <b>13</b> |
| <b>10 Building the Executable</b>                                   | <b>13</b> |
| <b>11 Versioning</b>                                                | <b>14</b> |
| <b>12 clautolisp Extensions</b>                                     | <b>14</b> |
| 12.1 Compositional list accessors (Cxxxr / Cxxxxr family) . . . . . | 14        |
| 12.2 ERROR . . . . .                                                | 15        |
| 12.3 LCM, LOGXOR . . . . .                                          | 15        |
| 12.4 VL-BT — full backtrace . . . . .                               | 15        |
| 12.5 VL-CATCH-ALL-ERROR-STACK — captured backtrace . . . . .        | 15        |
| 12.6 The CLAL- namespace (clautolisp-only helpers) . . . . .        | 16        |
| 12.7 Source . . . . .                                               | 19        |
| <b>13 See Also</b>                                                  | <b>19</b> |

## 1 Scope

This manual documents the `clautolisp` standalone executable: how to invoke it, what options it accepts, and what it does in each mode.

It does **not** document the AutoLISP / Visual LISP language. For the language reference — surface syntax, builtins, semantics, and host-product divergences — see the companion `autolisp-spec` subproject (sibling directory `../../autolisp-spec/`).

This document must be kept in sync with the implementation. Whenever `clautolisp/tools/clautolisp/source/main.lisp` changes its option parsing, banner, exit codes, or supported modes, this file must be updated in the same commit.

The rendered PDF (`user-manual.pdf`, sibling of this file) is also checked in. Whenever this Org file changes, run `make user-manual` (or `make documentation`) to regenerate the PDF and stage both files in the same commit.

## 2 Synopsis

```
clautolisp [options] FILE.lsp
clautolisp [options] -x EXPRESSION
clautolisp [options]                               (interactive REPL)
```

The implementation is shipped as two image-built executables, one per supported Common Lisp host:

- `clautolisp-sbcl` — built on SBCL via `make build-clautolisp-sbcl`.
- `clautolisp-ccl` — built on CCL via `make build-clautolisp-ccl`.

Both binaries are produced under `clautolisp/tools/clautolisp/bin/` and accept the same command-line surface. The rest of this manual treats them interchangeably and refers to the binary as `clautolisp`.

## 3 Modes of Operation

`clautolisp` has three mutually exclusive modes; the mode is determined by the operands. In every mode the runtime installs the core builtin function bindings before evaluation begins.

### 3.1 File mode

A non-option argument is treated as a path to an AutoLISP source file. The reader loads the file under the active dialect, the evaluator processes every form sequentially, and the binary exits once the file has been fully consumed.

```
clautolisp drawing-tools.lsp
clautolisp --bricscad path/to/script.lsp
```

### 3.2 Expression mode

The `-x EXPRESSION` option evaluates the supplied string as if it were the entire source of a file, then exits.

```
clautolisp -x "(setq x 7) (+ x 3)"
clautolisp --bricscad -x "(princ (atof \"0x1p4\"))"
```

The expression is read with the source name `<-x>`, which appears in diagnostic messages.

### 3.3 Interactive REPL

With no `FILE` and no `-x`, `clautolisp` drops into an interactive read-eval-print loop on standard input.

- Primary prompt: `~_ $ ~`
- Continuation prompt (printed when an expression is incomplete): three spaces.
- Each balanced top-level form is evaluated and the result is printed using the AutoLISP value formatter (the same formatter used by `prin1`).
- A reader or runtime error is reported as a one-line diagnostic; the REPL then re-prompts.
- Closing standard input (Ctrl-D on a fresh line) ends the REPL.

The REPL accepts multi-line input: as long as the parser reports `:unexpected-eof`, the REPL keeps reading continuation lines and appends them to the in-progress form before retrying.

## 4 Options

All options begin with two dashes except for `-x`. They may appear in any order before, after, or interleaved with the `FILE` argument; the last occurrence of an option wins. `--` is **not** a recognised end-of-options marker; the runtime forwards every CLI argument to the parser.

### 4.1 Dialect selection

The dialect descriptor controls reader strictness, host-specific permissive parsing, and the runtime knobs documented in `autolisp-spec`. The default is the conservative `strict` profile.

**--dialect NAME** Select the named dialect. NAME is one of `strict`, `autocad-2026`, `bricscad-v26`, `clautolisp`, `lax`.

**--strict** Equivalent to `--dialect strict`.

**--autocad** Equivalent to `--dialect autocad-2026`.

**--bricscad** Equivalent to `--dialect bricscad-v26`.

**--clautolisp** Equivalent to **--dialect clautolisp**.

**--lax** Equivalent to **--dialect lax**. Silences every encoding-dispatch diagnostic globally; useful when consuming code from multiple vendors. The runtime still honours each form; only the diagnostic is suppressed.

The **clautolisp** dialect includes ‘clautolisp extensions’ (eg. variadic functions).

Whatever the dialect selected, all the operators are available. Only those that are not specifically inside the set defined by the dialect produce warnings when compiled and when used.

Unknown dialect names produce a diagnostic on standard error and exit code 1.

## 4.2 Input selection

**-x EXPRESSION** Evaluate **EXPRESSION** instead of reading a file. Implies expression mode. Must be followed by exactly one argument.

## 4.3 REPL behaviour

**--quiet** Suppress the REPL banner; only the prompt is shown. No effect outside REPL mode.

## 4.4 Host backend

The Host Abstraction Layer (HAL, Phase 8 of the implementation roadmap) decides where CAD-host effects land. The dialect descriptor controls the **language**; the host backend controls the **target**. Both axes are independent.

**--host NAME** Select the backend. Currently accepted: **mock** (default since 0.0.14): an in-memory deterministic CAD-database substitute capable of **entget** / **ssget** / **getvar** / **getstring** etc. **null**: every CAD-host operation signals **:host-not-supported** — useful for programs that should not touch a host at all. **live** is reserved for Phase 16.

**--mock-input PATH** Attach the file at **PATH** as the MockHost’s prompt-stream. Lines are consumed in order by **getstring**, **getreal**, **getint**, **getpoint**, etc. so headless test rigs can feed deterministic answers to interactive scripts.

## 4.5 Colour output

`clautolisp` accents AutoLISP-symbol names in error messages, backtraces, and trace output with an ANSI colour when standard output is a terminal. The default is conservative (yellow on assumed-dark) and never fires when the output is being captured (pipe, file redirect, test harness).

**--no-color** Disable colour for this invocation. Equivalent to setting `$NO_COLOR` to any non-empty value (see <https://no-color.org>).

**\$NO\_COLOR** Universal no-color convention; honoured equivalently to **--no-color**.

**\$CLAUTOLISP\_SYMBOL\_FOREGROUND=NAME** Force the foreground colour for printed AutoLISP symbols. Accepted names (case and whitespace insensitive): `black`, `red`, `green`, `yellow`, `blue`, `magenta`, `cyan`, `white` and their `bright-` prefixed variants (`bright-black` is also `grey` / `gray`). `default` / `none` / `off` leaves the foreground axis bare.

**\$CLAUTOLISP\_SYMBOL\_BACKGROUND=NAME** Force the background colour for printed AutoLISP symbols. Same vocabulary as the foreground sibling; either or both env vars may be set. Both win over the automatic luminance probe so the user gets a deterministic look without depending on terminal detection.

**\$CLAUTOLISP\_BACKGROUND=dark|light** Override the terminal-background guess without specifying a colour. `dark` keeps the yellow accent; `light` flips it to blue.

**\$CLAUTOLISP\_PROBE\_OSC11** Opt-in to the active OSC 11 terminal query (spawns `/bin/sh + stty + dd` against `/dev/tty`). Off by default because on some platforms — notably macOS arm64 — the helper subprocess receives SIGTTOU and hangs CLI startup. Safe to set on most Linux desktops and iTerm2 / Terminal.app launched from a normal shell.

Example: yellow symbols on a dim-blue background, deterministically, no probing:

```
export CLAUTOLISP_SYMBOL_FOREGROUND=bright-yellow
export CLAUTOLISP_SYMBOL_BACKGROUND=blue
clautolisp -x '(foreach x t (princ x))'
```

## 4.6 Source-file and terminal encoding

- e ENC** Set the default source-file encoding for every `load` in the session, including nested (`load ...`) calls from init files. ENC is one of `utf-8`, `iso-8859-1`, `latin-1`, `windows-1252`, `cp1252`. Overrides the dialect default and the POSIX-locale guess (`LC_ALL` / `LC_CTYPE` / `LANG`). Surfaced to user code as the `*AUTOLISP-FILE-ENCODING*` global.
- E ENC** Declared terminal-IO encoding for the session. Surfaced to user code as the `*AUTOLISP-TERMINAL-ENCODING*` global so code that emits raw bytes can adapt. No stream rebinding is performed at the CL level — set this when your downstream tooling needs the declared encoding to match what you actually emit. Same ENC vocabulary as **-e**.

### 4.6.1 Launch-chain resolution and the degenerate case

At startup the runtime resolves the file encoding via `or-else(LC_ALL, LANG, LC_CTYPE, -e flag)` and pushes the result into two slots: `SYSCODEPAGE` (read-only host sysvar) and `*AUTOLISP-FILE-ENCODING*` (read-write global the user may `setq` at the REPL or in init files).

When **-e** is absent AND all three locale env vars are unset or empty (the **degenerate case** — common in CI containers and freshly-provisioned cloud hosts), the two slots diverge:

- `SYSCODEPAGE` / `DWGCODEPAGE` -> `"US-ASCII"`. The safest 7-bit assumption a vendor-compatible host sysvar can carry. User code that branches on (`getvar "SYSCODEPAGE"`) gets a non-empty deterministic value.
- `*AUTOLISP-FILE-ENCODING*` -> `"` (the **empty string**). The global is bound to a non-nil autolisp-string, but the empty value is a sentinel meaning "no preference — LOAD/OPEN should consult the dialect default." The runtime's `LOOKUP-AUTOLISP-FILE-ENCODING` helper treats empty the same way it treats an unbound global, so the LOAD precedence chain falls through to the dialect default: `--clautolisp` / `--autocad-2026` / `--bricscad-v26` -> `UTF-8`; `--strict` -> `ISO-8859-1`.

Observable from AutoLISP under the degenerate case:

```
*AUTOLISP-FILE-ENCODING*           ; => ""           (bound, non-nil)
(strlen *AUTOLISP-FILE-ENCODING*) ; => 0
```

```
(getvar "SYSCODEPAGE")           ; => "US-ASCII"
(load "lib/has-utf8-bytes")       ; loaded with UTF-8 under --clautolisp
```

Why the asymmetry: pinning `*AUTOLISP-FILE-ENCODING*` to "US-ASCII" in the degenerate case would make every source-file load strict ASCII regardless of dialect, and the first non-ASCII byte in any init file raises a stream decoding error. A 1-to-1 8-bit fallback (ISO-8859-1) would avoid the crash — every byte is a valid ISO-8859-1 character — but at the cost of silently mojibake'ing UTF-8 multi-byte sequences into pairs of Latin-1 characters. The empty-string sentinel cleanly hands the decision back to the dialect, which already declares what encoding its source files use.

The full design rationale is in `issues/closed/encoding-dispatch.issue` §"Degenerate launch case."

## 4.7 Informational

**--version** Print `clautolisp` <version> on standard output and exit 0.

The version is the value of `clautolisp.tools.clautolisp:*version*` (format `MAJOR.MINOR.DEVELOP`).

**--help** Print a short usage summary and exit 0.

## 5 Image-introspection variables (`*AUTOLISP-...*`)

At startup `clautolisp` publishes a set of globally-bound variables in the running AutoLISP image describing how it was invoked. User code can read them to branch on backend, dialect, encoding, etc. See `issues/closed/transmit-options.issue` for the full design and the alfe-side counterpart.

### 5.1 Static — bound once, before the first user action

**\*AUTOLISP-FRONTEND\*** The symbol `CLAUTOLISP` — `clautolisp` was the entry point. (Under alfe this is `ALFE`.)

**\*AUTOLISP-BACKEND\*** The symbol `CLAUTOLISP` — the engine running this code.

**\*AUTOLISP-DIALECT\*** The active dialect's canonical name as a symbol — `STRICT`, `AUTOCAD-2026`, `BRICSCAD-V26`.

**\*AUTOLISP-HOST\*** The HAL backend's name as a symbol — `MOCK-HOST`, `NULL-HOST`, or `nil`.



**\*AUTOLISP-MOCK-INPUT\*** Pathname (string) passed to `--mock-input`, or nil.

**\*AUTOLISP-GUI-COMMAND\*** Subprocess renderer command passed to `--gui`, or nil.

**\*AUTOLISP-TRACE\*** T when `--trace` was on at startup.

**\*AUTOLISP-FILE-ENCODING\*** Always a string (never nil). Resolved at launch from `-e ENC` (explicit), then the POSIX locale chain (`LC_ALL > LANG > LC_CTYPE`), falling back to the empty string "" when nothing resolves. The empty value is a sentinel — `LOAD / OPEN` treat it as "no preference" and fall through to the dialect default (UTF-8 for non-strict dialects, ISO-8859-1 for `--strict`). See "Launch-chain resolution and the degenerate case" under **Options**.

**\*AUTOLISP-TERMINAL-ENCODING\*** Always a string. Same resolution chain as **\*AUTOLISP-FILE-ENCODING\***, but falls back to "US-ASCII" rather than the empty string — terminal IO does not have a "dialect default" to fall through to.

**\*AUTOLISP-NO-INIT\*** T when `-norc / --no-init` was passed, or the env-var override suppressed init files.

**\*AUTOLISP-NO-COLOR\*** T when `--no-color` was passed.

**\*AUTOLISP-VERBOSE\*** T when `-v / --verbose` was passed.

**\*AUTOLISP-DEBUG\*** T when `-d / --debug` was passed.

**\*AUTOLISP-QUIET\*** T when `-q / --quiet` was passed.

**\*AUTOLISP-WILL-QUIT\*** Reserved for the `--quit` flag; always nil under clautolisp today. alfe sets it.

**\*AUTOLISP-MAIN\*** Reserved for the `--main` flag; always nil under clautolisp today.

**\*AUTOLISP-ACTIONS\*** Ordered list of the actions parsed off the command line, each shaped `(load "PATH")`, `(eval "TEXT")`, or `(interactive T)`. Empty list when the implicit REPL is the only action.

**\*AUTOLISP-HELP\*** String containing the `--help` output.

**\*AUTOLISP-VERSION\*** String like "1.0.47".

User code may `setq` any of these; the runtime does not re-read the values, so the modification is purely an information change visible to subsequent `~boundp~/value` lookups.

## 5.2 Dynamic — bound only while the matching action runs

**\*AUTOLISP-LOAD-PATHNAME\*** The pathname of the file being loaded by a `-l` action. Bound to the path string for the duration of the `load` call; reset to `nil` on return (success or signal).

**\*AUTOLISP-EXPRESSION\*** The source text of a `-x` action, bound while the expression is evaluated. Cleared on return.

**\*AUTOLISP-INTERACTIVE\*** `T` while the REPL is active. Cleared on REPL exit.

Code running inside an init file can therefore distinguish "loaded because of `-l init.lsp`" (`*AUTOLISP-LOAD-PATHNAME*` points at `init.lsp`) from "loaded as an init file from the standard search path" (`*AUTOLISP-LOAD-PATHNAME*` is `nil`).

## 6 Banner

In REPL mode (and only in REPL mode) `clautolisp` prints a one-line banner before the first prompt:

```
clautolisp 0.0.9 - REPL (STRICT dialect, null-host host) - Ctrl-D to exit.
```

The version string, dialect name, and host backend name reflect the current build and the selected `--dialect` / `--host`. Pass `--quiet` to suppress the banner.

## 7 Exit Codes

`clautolisp` uses the following exit codes. They are stable and may be relied upon by shell scripts.

- 0 Successful completion (file or `-x` ran without error; REPL exited cleanly via end-of-input; `--help` / `--version` printed and exited; an AutoLISP `exit` / `quit` termination was raised cleanly).

- 1 A runtime error escaped the AutoLISP error-handling layer, or the option parser rejected an unknown flag.
- 2 The requested file could not be opened (caught Common-Lisp `file-error`).

Within the REPL, individual evaluation errors are reported but do **not** terminate the loop; the binary will still exit 0 once stdin closes.

## 8 Errors and Diagnostics

- Reader errors are produced by the AutoLISP reader and surface as `simple-error` conditions whose printable form embeds a structured `diagnostic` (severity, code, message, source span). They are printed to standard error in batch modes and to standard error in REPL mode while the loop continues.
- Runtime errors are signalled as `autolisp-runtime-error` with a symbolic `code` (e.g. `UNDEFINED-FUNCTION`, `UNBOUND-VARIABLE`, `TYPE-ERROR`) and a human-readable message.
- Termination conditions raised by `exit` / `quit` (`autolisp-termination`) surface as a one-line notice on standard error and exit cleanly.

The exact set of error codes lives in the runtime; see `autolisp-runtime/source/api.lisp` for the canonical mapping.

### 8.1 Encoding-dispatch diagnostics (`enc-*`)

Each dialect dispatches a distinct diagnostic when user code uses an encoding-related extension that belongs to a different dialect. The diagnostics are **advisory**: the runtime always honours the form so user code stays runnable across dialects; the diagnostic just surfaces the portability concern.

Eight codes, all prefixed `enc-`:

| Code                                  | When                                                                                |
|---------------------------------------|-------------------------------------------------------------------------------------|
| <code>enc-foreign-dialect</code>      | a form belonging to a non-selected dialect                                          |
| <code>enc-extension-used</code>       | <code>--strict</code> only: any encoding extension                                  |
| <code>enc-unsupported-target</code>   | selected encoding has no host expression                                            |
| <code>enc-invalid-value</code>        | bad encoding name string                                                            |
| <code>enc-lispsys-out-of-range</code> | ( <code>setvar "LISPSYS" n</code> ) with <code>n</code> <code>\notin</code> {0,1,2} |
| <code>enc-codepage-mismatch</code>    | DWGCODEPAGE differs from SYSCODEPAGE                                                |
| <code>enc-unknown-codepage</code>     | unknown SYSCODEPAGE string                                                          |
| <code>enc-host-dependent</code>       | "ANSI" used in a write context (info)                                               |

Diagnostics print to standard error as `[enc-<code>] <message>`. The output stream is overridable for tests via the CL parameter `clautolisp.autolisp-runtime:*enc-diag`. Emission is silenced entirely by binding `*enc-diagnostic-suppress-p*` non-`nil` (the runtime side-door for the per-file pragma mechanism described below).

#### Two user-facing suppression knobs:

- **Per-code pragma.** Inside a source file:

```
(clal-suppress-enc-diagnostic 'enc-extension-used 'enc-foreign-dialect)
;; remainder of the file emits nothing for the two listed codes
```

Re-enable via `(clal-enable-enc-diagnostic 'enc-extension-used)` or, without arguments, `(clal-enable-enc-diagnostic)` to clear the entire suppression set. Code names accept either a string (`"enc-extension-used"`) or a symbol (`'enc-extension-used`); the leading `enc-` is optional.

- **-lax dialect.** Run `clautolisp` with `--lax` (or `--dialect lax`) to silence **every** encoding diagnostic globally for the session. Useful when consuming code from multiple vendors and you don't want to commit to one dialect's portability rules. The runtime still honours each encoding form; only the diagnostic is suppressed.

#### Static encoding-extension linter:

`(clal-lint-encoding-extensions FORM)` walks `FORM` recursively and emits the same `enc-*` diagnostics the runtime would, without actually evaluating any call. Useful for catching dead-code encoding extensions before they ship — pipe a file's parsed form-tree through the lint at compile time:

```
;; Hypothetical CI gate:
(clal-lint-encoding-extensions
```

```

(read-from-string-or-something-like-it
 (slurp "lib/encoding-aware.lsp"))))
;; -> [enc-extension-used] LOAD's positional [encoding] argument...
;;    [enc-foreign-dialect] OPEN CCS-SUFFIX ...

```

The lint shares SIGNAL-ENCODING-DIAGNOSTIC with the runtime emitters, so dialect / pragma / -lax gates apply identically. Returns T when the walk completes.

Currently wired:

- LOAD with the positional [encoding] argument under --strict (enc-extension-used), --autocad / --bricscad (enc-foreign-dialect), silent under --clautolisp.
- OPEN encoding forms, all three routed:
  - positional "utf8" / "utf8-bom" — native under --autocad; foreign under --bricscad / --clautolisp; extension-used under --strict.
  - positional "UTF-8" / "UTF-8-BOM" / "CP-NNNN" / ... — native under --clautolisp; foreign under --autocad / --bricscad; extension-used under --strict.
  - ,ccs=ENC mode-suffix — native under --bricscad; foreign under --autocad / --clautolisp; extension-used under --strict. The suffix is stripped from the mode-string before CL OPEN sees it.
- GETVAR / SETVAR on LISPSYS under every dialect except --autocad (the native dialect of LISPSYS):
  - --strict -> enc-extension-used.
  - --bricscad / --clautolisp -> enc-foreign-dialect with the dialect sub-tag.
- SETVAR "LISPSYS" <n> with n \notin {0,1,2} emits enc-lispsys-out-of-range regardless of dialect (the spec mandates the domain). The write proceeds — matching the vendor "permissive but warn" behaviour.
- OPEN with an encoding the active host can't natively express (e.g. "UTF-16-LE" under --autocad) emits enc-unsupported-target. Informational; the runtime still attempts the open.

- OPEN in WRITE direction with "ANSI" / "MBCS" emits `enc-host-dependent` (info-level): output resolves through the host's SYSCODEPAGE at I/O time and isn't portable across hosts. Reading under those encodings is silent.
- The internal SET-DRAWING-CODEPAGE wrapper (CL-level; hook for future drawing-load paths) emits `enc-codepage-mismatch` when the incoming DWGCODEPAGE canonical form differs from SYSCODEPAGE — catches the "Czech-authored drawing on a French host" failure mode before string decode time.
- `%CANONICAL-CODEPAGE-STRING` emits `enc-unknown-codepage` when asked to canonicalise a spelling not in the known table (typos, unsupported vendor aliases). The raw value passes through.

The full set of codes, the per-dialect dispatch table, and the roadmap for the remaining hooks (OPEN, BricsCAD ,ccs= parsing, DWGCODEPAGE mismatch detection at drawing-load time) is in `issues/closed/encoding-dispatch.issue`.

## 9 Examples

```
# Print a literal symbol.
$ clautolisp -x "(print 'hi)"
HI

# Evaluate a small program under the BricsCAD V26 dialect.
$ clautolisp --bricscad -x "(setq x 1) (+ x 2)"

# Run a script file under the AutoCAD 2026 dialect.
$ clautolisp --autocad some-script.lsp

# Interactive REPL, default dialect.
$ clautolisp
clautolisp 0.0.1 - REPL (STRICT dialect) - Ctrl-D to exit.
_$ (setq xs (list 1 2 3))
(1 2 3)
_$ (length xs)
3
_$ ^D
```

```
# Interactive REPL with no banner.
$ clautolisp --quiet

# Show the version and exit.
$ clautolisp --version
clautolisp 0.0.1
```

## 10 Building the Executable

From the `clautolisp/` subproject root:

```
make build-clautolisp-sbcl # produces tools/clautolisp/bin/clautolisp-sbcl
make build-clautolisp-ccl  # produces tools/clautolisp/bin/clautolisp-ccl
```

Both targets reuse the `read-autolisp` build harness. The SBCL build sets `:save-runtime-options t` so the saved core forwards every command-line argument to `clautolisp` rather than letting SBCL intercept `--help` or other runtime flags.

## 11 Versioning

The version string is stored in `clautolisp/tools/clautolisp/source/version.lisp` as `*version*`. The format is `MAJOR.MINOR.DEVELOP`. The `DEVELOP` counter is bumped on every change that touches `clautolisp` source code; the running binary's `--version` output is therefore a stable identifier of the build.

## 12 clautolisp Extensions

The core builtin registry installed by `clautolisp` is a **superset** of the operators listed in the local AutoLISP / Visual LISP specification draft. The extras documented in this section are convenience extensions and compatibility helpers that real-world AutoLISP code relies on without going through the spec process.

Code that targets the strict shared standard (the AutoCAD/BricsCAD intersection) **MUST NOT** depend on the operators below; tests written against `autolisp-test/harness/inventory.sexp` are intentionally restricted to the spec, and the inventory does not list these names.

## 12.1 Compositional list accessors (Cxxxr / Cxxxxr family)

The local specification documents CAR, CDR and CADR. Beyond those, clautolisp also installs:

**two-level** CAAR, CDAR, CDDR

**three-level** CAAAR, CAADR, CADAR, CADDR, CDAAR, CDADR, CDDAR, CDDDR

**four-level** CAAAAAR, CAAADR, CAADAR, CAADDR, CADAAR, CADADR, CADDAR, CADDDR, CDAAAR, CDAADR, CDADAR, CDADDR, CDDAAR, CDDADR, CDDDR, CDDDDR

Each composition delegates to CAR / CDR so a non-list argument anywhere along the chain still surfaces a CAR-shaped or CDR-shaped runtime error under the proper operator name.

```
$ clautolisp -x "(caddr ' (a b c d e))"
D
```

## 12.2 ERROR

(error MESSAGE-OR-STRING ...) raises an AutoLISP-visible runtime error with code :user-error and the supplied message. Modeled after the common Visual LISP idiom of calling (error ...) to abort with a user-facing message, and catchable with vl-catch-all-apply.

```
$ clautolisp -x "(vl-catch-all-error-message
                  (vl-catch-all-apply 'error '(\\"oops\\")))"
"oops"
```

## 12.3 LCM, LOGXOR

Convenience numerics that complement the in-spec GCD, LOGAND and LOGIOR. LCM takes two or more integers and returns their least common multiple; LOGXOR takes two or more integers and returns their bitwise exclusive-or.

```
$ clautolisp -x "(lcm 4 6)"
12
$ clautolisp -x "(logxor 12 10)"
6
```



## 12.4 VL-BT — full backtrace

The local specification reserves VL-BT as a Visual LISP backtrace operator. `clautolisp` implements it as a real backtrace printer rather than a stub: `(vl-bt)` prints the AutoLISP call stack at the current evaluation point, one line per frame, most recent first.

The runtime tracks the stack through `*autolisp-call-stack*`, a dynamically-bound list of evaluator frames pushed by `autolisp-eval` and by every SUBR / USUBR call. Frame kinds are `:eval`, `:special-op`, `:subr` and `:usubr`.

## 12.5 VL-CATCH-ALL-ERROR-STACK — captured backtrace

`(vl-catch-all-error-stack OBJECT)` is a `clautolisp`-only extension companion to the standard `vl-catch-all-error-message`. Given a catch-all error object, it returns the AutoLISP call-stack snapshot captured at the time the underlying `autolisp-runtime-error` was raised. Each frame is a cons (KIND . PAYLOAD), where PAYLOAD is either the form being evaluated (for `:eval` / `:special-op`) or the cons (NAME . ARGS) of the function call (for `:subr` / `:usubr`).

This is the operator the `autolisp-test` harness calls to render an AutoLISP backtrace inside a FAIL detail when a tested function raises. AutoCAD and BricsCAD do not expose the captured stack, so the harness gracefully falls back to message-only reporting on those hosts.

```
$ clautolisp -x "(setq c (vl-catch-all-apply 'car '(7)))
                  (length (vl-catch-all-error-stack c))"
;; one or more frames captured at the time CAR was rejected
3
```

Code that needs to remain portable should test for the operator before calling it:

```
(if (boundp 'vl-catch-all-error-stack)
    (vl-catch-all-error-stack object)
    nil)
```

## 12.6 The CLAL- namespace (clautolisp-only helpers)

The four other sub-sections above (`Cxxxxr` accessors, `ERROR`, `LCM`~/~`LOGXOR`, `VL-BT`~/~`VL-CATCH-ALL-ERROR-STACK`) cover extensions that complete an

established AutoCAD/BricsCAD idiom without inventing a new one. From clautolisp 1.0.76 onward, a second class of extensions ships under a reserved **CLAL-** prefix: helpers that have **no** vendor counterpart and that we want to make trivially distinguishable from any future Autodesk- or Bricsys- documented name. Code that calls one of these functions is explicitly opting into the clautolisp dialect.

The probing convention for portable code:

```
(if (boundp 'clal-sysvar-list)
    (clal-sysvar-list)
    ;; vendor fallback -- there isn't one; either accept that the
    ;; query is unavailable or fall back to the SETVAR command.
    nil)
```

### 12.6.1 CLAL-SYSVAR-LIST – list every sysvar known to the host

(clal-sysvar-list) returns a list of strings: the sysvar names the active host (mock or live) reports, sorted in case-sensitive ASCII order. On a default MockHost the list has 1836 entries – the same universe as the autolisp-spec §16 **System Variables** catalogue.

```
$ clautolisp -x "(length (clal-sysvar-list))"
1836
```

```
$ clautolisp -x "(car (clal-sysvar-list))"
"3DCONVERSIONMODE"
```

### 12.6.2 CLAL-SYSVAR-APROPOS – substring search over sysvar names

(clal-sysvar-apropos PATTERN) returns the subset of (clal-sysvar-list) whose names contain PATTERN as a case-insensitive substring. Useful at the REPL when you remember "there's something with ANG in it" but not the exact name.

```
$ clautolisp -x "(clal-sysvar-apropos \"ANG\")"
("ANGBASE" "ANGDIR" "INPUTHISTORYMODE" ...)
```

```
$ clautolisp -x "(clal-sysvar-apropos \"ang\")"      ;; case-insensitive
("ANGBASE" "ANGDIR" "INPUTHISTORYMODE" ...)
```

```
$ clautolisp -x "(clal-sysvar-apropos \"ERRNO\")"
```

```
("ERRNO")
```

```
$ clautolisp -x "(length (clal-sysvar-apos \\"))" ;; empty -> all  
1836
```

A non-string PATTERN raises `:invalid-argument`. A pattern that matches nothing returns `nil`.

### 12.6.3 CLAL-SYSTEM-CODEPAGE – canonical SYSCODEPAGE

`(clal-system-codepage)` wraps `(getvar "SYSCODEPAGE")` and returns the canonical clautolisp codepage spelling. Vendor "ANSI\_NNNN" and clautolisp "WINDOWS\_NNNN" inputs both fold onto "CP\_NNNN"; Unicode names ("UTF-8", "ISO-8859-1", "US-ASCII") pass through; the pre-Phase-1 empty placeholder becomes "ANSI".

```
LC_ALL=fr_FR.WINDOWS-1252 clautolisp -x "(clal-system-codepage)"  
"CP-1252"
```

```
LC_ALL=en_US.UTF-8 clautolisp -x "(clal-system-codepage)"  
"UTF-8"
```

### 12.6.4 CLAL-DRAWING-CODEPAGE – canonical DWGCODEPAGE

`(clal-drawing-codepage)` is the same canonicalisation applied to `(getvar "DWGCODEPAGE")`. At launch DWGCODEPAGE defaults to SYSCODEPAGE; loading a drawing that carries its own codepage overrides it.

### 12.6.5 CLAL-CODEPAGE-MISMATCH-P – defensive comparison

`(clal-codepage-mismatch-p)` returns T when the canonical SYSCODEPAGE and DWGCODEPAGE differ, `nil` otherwise. Cheap defensive check before loading drawing-authored text: a mismatch means strings authored under one ANSI codepage may garble when interpreted under another.

The comparison runs on the canonicalised form, so "ANSI\_1252" vs "WINDOWS-1252" correctly registers as no mismatch.

### 12.6.6 CLAL-FILE-ENCODING – BOM-sniffing file-encoding probe

`(clal-file-encoding PATH)` opens PATH, reads the first four bytes, and returns the canonical clautolisp encoding implied by the byte-order mark:

| BOM bytes   | Returned string |
|-------------|-----------------|
| EF BB BF    | "UTF-8-BOM"     |
| FF FE       | "UTF-16-LE"     |
| FE FF       | "UTF-16-BE"     |
| FF FE 00 00 | "UTF-32-LE"     |
| 00 00 FE FF | "UTF-32-BE"     |
| no BOM      | "ANSI"          |

(UTF-32-LE / UTF-16-LE share the same two leading bytes; `CLAL-FILE-ENCODING` checks UTF-32 first so the two-byte UTF-16 prefix doesn't mask a four-byte UTF-32 BOM.)

Mirrors BricsCAD's `VLE-FILE-ENCODING` but preserves the UTF-8-BOM / plain-UTF-8 distinction the BricsCAD function flattens. A nonexistent `PATH` returns `nil` and sets `ERRNO 73`, matching `LOAD`'s missing-file behaviour.

```
$ clautolisp -x "(clal-file-encoding \"~/.autolisp\")"
"ANSI"
```

The full normative entries for these functions live in `autolisp-spec §16 clautolisp Extensions`; this section is a quick-reference summary.

## 12.7 Source

The implementation of every operator listed above lives in:

- `clautolisp/autolisp-builtins-core/source/api.lisp` — registration and per-builtin code.
- `clautolisp/autolisp-builtins-core/README.org` — module-level rationale, kept in sync with this manual section.
- `clautolisp/autolisp-runtime/source/api.lisp` — backtrace tracking (`*autolisp-call-stack*`, `current-autolisp-call-stack`, the call-stack slot on `autolisp-runtime-error`, and the `autolisp-catch-all-error-call-stack` accessor).

## 13 See Also

- `../.. / autolisp-spec /` — AutoLISP / Visual LISP language reference, including dialect divergences and the normative source for builtin semantics.

- `encoding-migration.org` — Vendor-to-clautolisp migration notes for file-encoding extensions (AutoCAD LISP/SYS / third-arg `open`, BricsCAD `,ccs=` / `vle-file-encoding`).
- `PLAN.md` — Implementation roadmap and per-phase completion notes.
- `clautolisp/autolisp-reader/` — Reader subsystem (lexer, parser, diagnostics, dialect descriptors).
- `clautolisp/autolisp-runtime/` — Runtime object model and evaluator.
- `clautolisp/autolisp-builtins-core/` — Core builtin function registry (the layer that supplies `print`, `+`, `atof`, etc. consumed by the CLI).
- `../../autolisp-test/` — Conformance test suite. Uses `vl-catch-all-error-stack` and `vl-bt` to render AutoLISP backtraces inside FAIL details when a tested function raises on clautolisp.